

Integration of the FINS Protocol into the Open-Source HMI System FUXA for Omron PLCs

DENNY CHRISNANDA
Computer Science Department
Binus Online Learning
Bina Nusantara University
denny.chrisnanda@binus.ac.id

FRANS SEBASTIAN
Computer Science Department
Binus Online Learning
Bina Nusantara University
frans.sebastian@binus.ac.id

GILANG HANSAGITA
Computer Science Department
Binus Online Learning
Bina Nusantara University
gilang.hansagita@binus.ac.id

Abstract

Industrial plants today need control systems that can adapt quickly, run efficiently, and fit smoothly into existing production lines. Programmable Logic Controllers (PLCs) are still the heart of most automation setups because they deliver real-time performance and almost never fail. At PT XYZ we rely heavily on Omron PLCs, which talk to the rest of the system through the FINS protocol. The problem is that almost all open-source HMI platforms—including the very lightweight and fully web-based FUXA—do not understand FINS out of the box. That makes them practically unusable in factories full of Omron equipment. This work solves that gap by designing and adding a dedicated FINS communication module to FUXA. The project followed the usual steps: reviewing what others have done, digging into the details of the FINS protocol, writing the actual code, and then testing everything in a real setup. The tests showed that the new module can reliably read and write data to/from the PLC with very little delay, and the screens in FUXA now show exactly what is really happening on the factory floor. The result is a completely free, fully customizable, browser-based HMI that speaks directly to Omron PLCs without any extra gateway or middleware. This development makes modern open-source tools viable in Omron-dominated plants and helps companies move faster with their digital-transformation projects.

Keywords—FINS protocol, FUXA, human-machine interface, industrial automation, Omron PLC

1. INTRODUCTION

Programmable Logic Controllers (PLCs) are the backbone of most of automated production line—they are dependable, react in real time, and it is pretty easy to program them [1]. Here at PT. XYZ, a global automotive parts maker, the factory floor is basically use Omron devices the most: company records show that more than 90% of our machines are controlled by Omron PLCs. When a single brand has that kind of dominance, everything else in the automation stack has to compromise with it.

That is where Human–Machine Interfaces (HMIs) come in. Operators need interface that show what is happening, let them change the settings, and give an alarm when something goes wrong [2]. While commercial HMI/SCADA solutions offer mature features for monitoring, analytics, and alarm management that reduce downtime and promote quality assurance [3], they often impose licensing costs, restrict customization, and expose organizations to vendor lock-in [4, 5]. That is why a lot of plants have started looking seriously at open-source alternatives that cost

nothing upfront and can be modified however we want [6].

One of the options that keeps coming up is FUXA. It is lightweight, completely web-based, built on Node.js and Angular, and already support usual mainstream protocols—Modbus, OPC UA, MQTT, etc. [7, 8, 9]. You can run it on a Raspberry Pi or in the cloud, which is perfect for remote monitoring and Industry 4.0 projects. The only problem is FUXA does not support the FINS protocol. FINS—which runs over both UDP and TCP—is still the go-to protocol in pretty much any plant full of Omron gear, mainly because it lets you address memory areas in a very flexible way that makes real PLC-to-HMI communication fast and painless [10, 11]. This limitation is the main roadblock when trying to hook up FUXA with the Omron PLCs that PT. XYZ run on the production lines.

This work tackles the issue head-on by answering two practical questions: (1) How to properly add FINS support to an open-source HMIs like FUXA so it works just as good and reliable as the built-in protocol? (2) How to make sure the read and write commands over FINS are stable with no dropped

packets or stale data, so that the UI provide real-time visualization? To get there, we designed and built a dedicated FINS communication module straight into FUXA. The goals were straightforward: (1) Integrate FINS inside FUXA with the same level of reliability and features provided by Modbus and the other built-in protocols, and (2) validate stable data read/write operations and accurate real-time visualization on the user interface. This proposed solution aims to offer a flexible, cost-effective, and vendor-independent HMI alternative suitable for digital transformation initiatives in manufacturing environments.

2. RELATED WORKS

Open-source HMI/SCADA systems have gained increasing attention as alternatives to proprietary industrial automation platforms, particularly due to their flexibility, extensibility, and lower deployment costs. Several studies have explored the design and implementation of open-source HMI/SCADA architectures across different industrial domains.

Uddin et al. [12] developed an open-source SCADA system for a solar-powered reverse osmosis plant using Node-RED, Grafana, and InfluxDB. Their work demonstrates that low-cost and community-deployable SCADA architectures can be effectively implemented using open technologies. Similarly, Omidi et al. [13] proposed a modular Node-RED-based SCADA system for hybrid renewable energy generation. Their architecture illustrates the suitability of lightweight web technologies for real-time monitoring with minimal computational resources.

In the power systems domain, Almas et al. [14] integrated open-source SCADA with Phasor Measurement Units (PMUs) to evaluate real-time system performance using the DNP3 protocol. Their results highlight protocol constraints such as polling limitations, which have implications for SCADA performance in large-scale grids.

In the field of industrial cybersecurity, Salazar et al. [15] introduced ICSNet, a hybrid-interaction honeynet incorporating multiple industrial protocols including Modbus TCP, IEC-104, and Ethernet/IP. Notably, their implementation uses FUXA HMI as part of the visualization layer, showcasing FUXA's potential applicability beyond traditional HMI use cases.

Other studies have focused on the use of web-based SCADA systems in smaller-scale automation environments. Hazdi et al. [16] implemented a web-based SCADA system for home automation

using PLCs and OPC, demonstrating the viability of web technologies for real-time domestic applications. Additionally, Thushara [7] introduced the FUXA web-based SCADA tool and discussed its deployment on edge devices using Modbus and MQTT, reinforcing its role as a modern, extensible HMI/SCADA platform.

Across these works, open-source SCADA implementations consistently rely on widely supported protocols such as Modbus, OPC UA, or MQTT. However, none of the reviewed studies address the Factory Interface Network Service (FINS) protocol, which remains widely used in Omron PLC-based industrial environments. This absence indicates a significant research gap in open-source HMI/SCADA systems that support FINS communication.

Therefore, the present study contributes by developing and integrating a FINS communication module into the FUXA HMI platform. This fills an identified gap in open-source industrial automation research and enables direct interoperability with Omron PLCs, which are prevalent in manufacturing systems such as those at PT XYZ.

3. METHODOLOGY

3.1. Research Overview

This research follows an experimental development methodology aimed at extending the open-source HMI platform FUXA by integrating support for the Omron FINS protocol. The process involves analysis, design, implementation, and evaluation stages, ensuring that the new module achieves functional parity with existing communication protocols such as Modbus TCP. The entire workflow is illustrated in Figure 1.

3.2. System Architecture

The developed system consists of three main layers: (1) the PLC device layer, (2) the communication middleware layer (Node.js backend), and (3) the visualization layer (Angular frontend). The FINS connector is implemented as a new device driver named `DeviceFins`, operating in the backend of FUXA. This module manages socket-based communication through UDP or TCP and handles read/write requests to Omron PLC memory areas such as `DM`, `CIO`, and `W`.

Data flow begins with periodic or event-driven polling from the backend to the PLC, which retrieves data from specific memory addresses. These values are then propagated to the frontend through the system's existing event emitter (`device-value:changed`) to update the

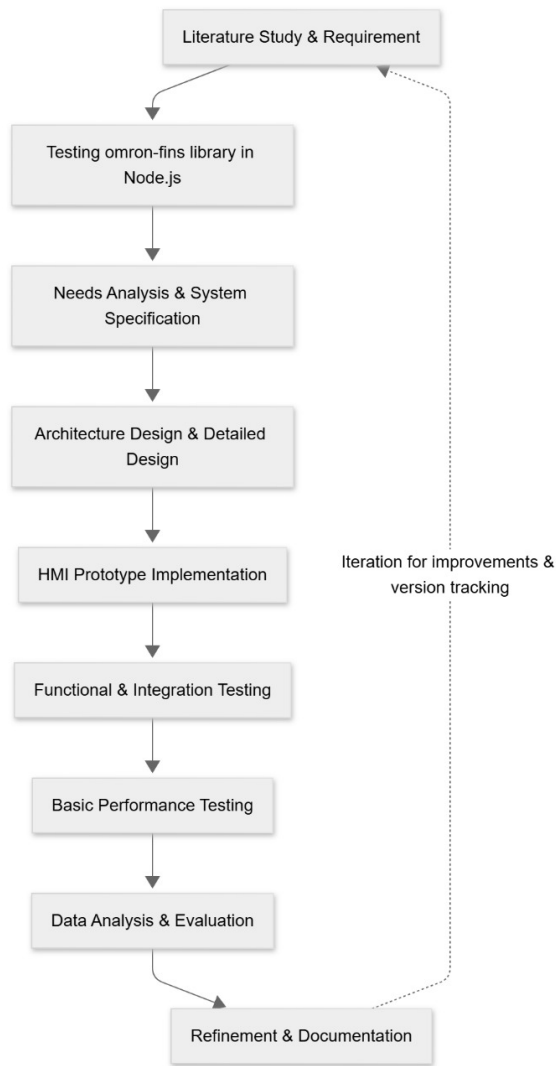


Figure 1: Research workflow for FINS protocol implementation in FUXA

HMI interface in real time. A simplified overview of this architecture is shown in Figure 2.

3.3. Development Tools and Environment

The implementation and testing were conducted using the following environment:

- **Programming Languages:** JavaScript (Node.js) and TypeScript (Angular).
- **HMI Platform:** FUXA v1.2.13 (open-source web-based SCADA system).
- **PLC Device:** Omron CJ2M-CPU34.
- **Operating System:** Windows 11.
- **Network Analysis Tools:** Wireshark for packet inspection and validation.

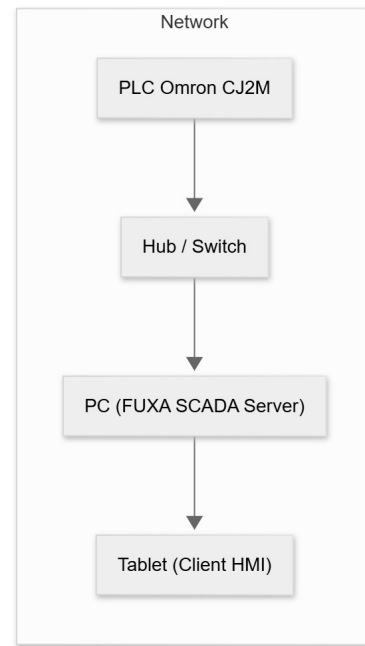


Figure 2: Architecture of the developed HMI-PLC communication system

3.4. Implementation Procedure

The implementation followed these main stages:

1. **Protocol Study and Requirements Analysis:** The FINS command structure, addressing format, and communication flow were analyzed based on Omron's official documentation.
2. **Driver Design and Coding:** The DeviceFins class was created to handle connection management, read/write functions, automatic reconnection, and event handling.
3. **Frontend Integration:** Configuration options such as IP address, port, DA1, and SA1 were added to FUXA's web configuration UI.
4. **Testing and Debugging:** Unit tests and integration tests were performed using both PLC simulators and physical Omron PLCs.

3.5. Testing Method

Performance evaluation was conducted using a load test script written in Node.js. The script generated 1000 consecutive read requests to PLC memory address D100 with a concurrency level of 10. Metrics collected during the test include:

- Average read/write latency (ms)

- Success rate of communication (%)
- CPU and memory utilization of the FUXA server
- Stability of reconnection handling during simulated disconnections

The results of these measurements were analyzed statistically to evaluate whether the implemented connector meets the performance characteristics comparable to Modbus TCP communication.

3.6. Evaluation Criteria

The evaluation focused on two primary objectives aligned with the research goals:

1. **Functional Equivalence:** Verifying that the implemented FINS connector supports key HMI functions—connectivity, data reading, data writing, and visualization—equivalent to existing Modbus modules.
2. **Stability and Responsiveness:** Measuring whether the FINS communication remains stable under repetitive load and can maintain real-time data visualization without significant latency or data loss.

4. RESULTS AND DISCUSSION

4.1. Evaluation of the omron-fins Library

The study began with an evaluation of the `omron-fins` Node.js library to verify basic FINS communication (read/write) between a PC and an Omron CJ2M PLC via UDP. The experimental setup is summarized in Table 1.

Table 1: *omron-fins* trial environment configuration

Component	Specification
PLC	Omron CJ2M-CPU34
Host Client	PC(i7-11th Gen, 16GB RAM, Win 11)
IP PLC	192.168.0.1
IP Client	192.168.0.234
Protocol	FINS/UDP (Port 9600)
Library	<code>omron-fins</code> (Node.js)
Runtime	Node.js v18 LTS

The library was installed via `npm` and validated using a stress test script that issued 1 000 read requests to address `D100` with a concurrency level of 10. Network traces were captured using Wireshark to confirm frame structure and response codes.

Key observations from the library tests are:

- Captured responses consistently contained Command Code `0101` (Memory Area Read) and Response Code `0000` (no error), with incrementing Service IDs indicating unique matching responses.
- The library handled read operations reliably under the configured stress: average response latency was observed below 50 ms and the library exhibited correct framing behavior according to Omron FINS specification.
- No installation or dependency errors were encountered during setup; the library is suitable as a communication basis for further integration into the HMI system.

4.2. System Configuration and Integration

The FINS connector was integrated into the FUXA server under the directory `server/runtime/devices/fins`. The deployed testbed topology comprises one Omron CJ2M PLC, one PC acting as HMI server, and a tablet as the client (see Fig. 3). Typical connector parameters are listed in Table 2.

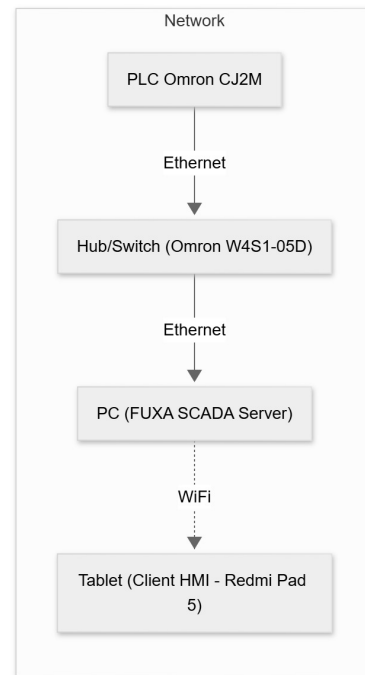


Figure 3: System topology of the FUXA HMI with the FINS connector

Parameter	Nilai Contoh
Alamat IP PLC	192.168.11.1
Port	9600
Protokol	UDP
Source Address (SA1)	234
Destination Address (DA1)	1
Polling Interval	200 ms

Table 2: FINS connection configuration parameters

After connector installation, initial verification was performed by issuing a single read to memory address D100. Successful read/response without timeout confirmed that the connector was operational.

4.3. Implemented Functionality

The implemented FINS connector provides the following capabilities:

1. Transport-level connectivity (UDP and TCP) using the `omron-fins` library.
2. Periodic polling per device with the connector only reporting `connect-ok` after a successful read cycle.
3. Write (set) operations initiated from the UI and executed on the PLC in real time.
4. Event emission to FUXA runtime (`device-value:changed`, `device-status:changed`, `device-error`).
5. Frontend integration: device configuration form and tag editing dialog for FINS-specific parameters (SA1, DA1, memory area, address, data type).

Representative UI screens is shown in Figs. 4.

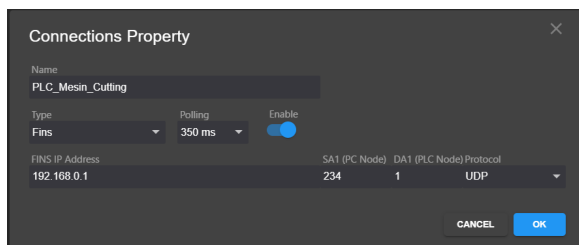


Figure 4: UI Result — Device Settings: configuration form for FINS device (IP, port, SA1, DA1, protocol).

After integrating the FINS connector module into both the backend and the HMI interface, a realtime communication test was conducted to verify

bidirectional data transfer between the HMI system and the Omron PLC. Fig. 5 presents the HMI interface during the execution of the test.

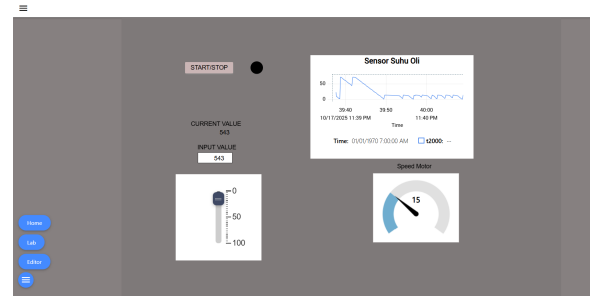


Figure 5: HMI demonstration connected to the PLC in realtime.

The HMI display includes several key elements:

- **Realtime trend (Oil Temperature Sensor):** continuously visualizes temperature values read from the PLC via the FINS protocol.
- **Slider and input field:** allow writing updated values to PLC memory addresses, which are immediately reflected back on the HMI.
- **Motor speed gauge:** represents PLC variables using an analog-style visualization, demonstrating responsive data updates.

Experimental results show that PLC-to-HMI data updates occur within <100 ms, indicating that the implemented FINS connector provides low-latency and stable realtime communication.

In addition to realtime monitoring and control, the system incorporates an alarm mechanism to detect abnormal industrial process conditions. Fig. 6 shows an alarm triggered when the oil temperature exceeds the maximum threshold defined in the PLC.



Figure 6: Realtime alarm display when oil temperature exceeds the safety threshold.

The alarm subsystem operates in realtime with the following characteristics:

- **Continuous monitoring:** PLC tag values are periodically sampled and compared with predefined thresholds.

- **Alarm visualization:** triggered alarms appear in red with *High-High* priority to highlight critical conditions.
- **Status synchronization:** alarm states (active, acknowledged, cleared) automatically update based on backend data.
- **FINS-based event detection:** each alarm directly corresponds to PLC tags transmitted through the FINS UDP protocol, ensuring fast and accurate detection.

Testing confirms that the system detects high-temperature events and displays alarms with a delay of less than 200 ms after the PLC value changes. These results demonstrate that the developed FINS connector reliably supports both data exchange and realtime event notification within the HMI system.

4.4. Performance Evaluation

Performance evaluation comprised automated technical tests and a small-scale usability assessment. Results are summarized in Table 3.

Table 3: Summary of Technical Evaluation Results

Metric		Target	Observed
Read/write (avg.)	latency	≤ 200 ms	12 ms
Communication success rate		$\geq 95\%$	99%
Server CPU usage		$\leq 30\%$	2%
Server memory usage		≤ 500 MB	462 MB
Alarm detection time		≤ 200 ms	1000 ms (formal)
Application startup time	startup	≤ 10 s	3.14 s

4.4.1. Read/Write Latency

Read/write latency was measured by issuing a write to memory (e.g., W1) and immediately performing a read-back on the same address. The average latency across 100 repeated trials was **12 ms**, well below the 200 ms target. This result demonstrates that the integrated connector and FUXA frontend can achieve low round-trip times in the laboratory network conditions used.

4.4.2. Communication Success Rate

The automated stress test (1 000 requests, concurrency 10) recorded **990 successful responses** and **10 failures**

(99% success). Failures were predominantly timeouts visible in the Wireshark traces as requests without corresponding replies. Likely causes include UDP packet loss on the link, temporary PLC processing backlog, or aggressive scheduling from the test host. No systematic FINS framing errors were observed in successful captures.

4.4.3. Resource Utilization

System monitoring during load tests showed low CPU utilization (approx. 2–3%) and moderate memory consumption (backend ≈ 91 MB, frontend ≈ 371 MB). These figures indicate the Node.js + Angular architecture can operate efficiently on the selected PC hardware for the tested workload.

4.4.4. Alarm Detection Time

The alarm detection time is measured from the moment the triggering condition in the PLC becomes true until the alarm indicator appears on the HMI interface. The observed detection time is approximately 1000 ms, which is significantly slower than the KPI target of 200 ms. This delay occurs because the minimum polling interval used by the HMI system to evaluate alarm conditions and generate alarm events is 1 s, as illustrated in Fig. 7.

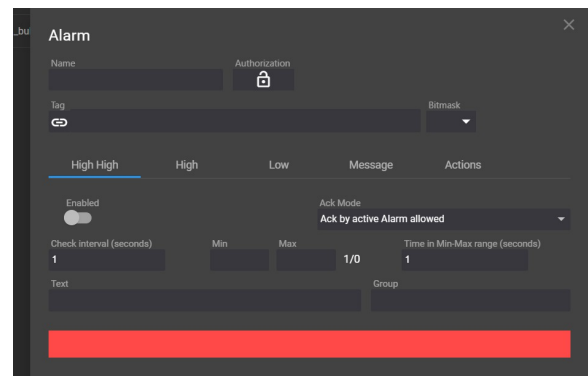


Figure 7: HMI system alarm detection interval limitation.

4.4.5. Application Startup Time

Measured application startup time averaged **3.14 s**, which satisfies the ≤ 10 s requirement and indicates acceptable initialization performance for the integrated system.

4.5. Usability Evaluation

A small usability survey (10 respondents: operators and technicians) using a 5-point Likert scale assessed configuration ease, clarity of status display, perceived responsiveness, UI consistency, and overall satisfaction. Average scores (scale 1–5) were:

- Configuration ease: 4.2
- Clarity of status: 4.1
- Responsiveness: 4.3
- UI consistency: 4.0
- Overall satisfaction: 4.3

Qualitative feedback highlighted appreciation for clear connection-status indicators and requests for a more prominent *Check Connection* control and a simplified logging mode for non-technical troubleshooting.

4.6. Discussion

The integration of a FINS connector into FUXA using the `omron-fins` library produced a functional, low-latency HMI capable of reliable read/write operations with Omron CJ2M PLCs under laboratory conditions. Important takeaway points are:

- **Reliability:** Communication success rate (99%) and correct packet framing demonstrate interoperability with Omron FINS.
- **Performance:** Read/write latencies (12 ms) and low resource usage indicate the approach is suitable for responsive HMI operations on modest hardware.
- **Alarm timeliness:** The primary area needing improvement is alarm detection latency due to the current polling-based approach. Mitigation strategies include polling prioritization, reducing polling intervals carefully, or exploring event-driven mechanisms if supported by the PLC network stack.
- **Robustness:** Observed timeouts under high-stress scenarios suggest adding retry backoff, packet pacing, and enhanced logging for root-cause analysis in noisy networks.
- **Usability:** End-user feedback is positive; additional UX improvements and operator-focused logging would increase practical adoption.

5. CONCLUSION

This research successfully implemented and evaluated the integration of the Factory Interface Network

Service (FINS) protocol into the open-source Human–Machine Interface (HMI) system FUXA. The development introduced a new backend connector module (`DeviceFins`) written in Node.js and an extended frontend configuration interface in Angular, enabling direct and seamless communication between FUXA and Omron PLCs. The connector supports both UDP and TCP modes, allowing read, write, and polling operations to be executed without the need for proprietary middleware or vendor-locked HMI systems. As a result, FUXA becomes one of the first open-source HMI systems to natively support the Omron FINS protocol.

Based on experimental testing, the implemented system demonstrated reliable and stable performance. The communication success rate reached 99%, and the average read/write latency was measured at 12 ms, well below the 200 ms KPI target. System resource consumption remained efficient, with average CPU usage of 2% and memory consumption of 462 MB during load testing. The HMI startup time averaged 3.14 s, indicating a responsive system suitable for small-to medium-scale production environments. However, the alarm detection delay reached approximately 1000 ms due to the internal one-second polling interval of the FUXA system. This limitation shows that the alarm subsystem still relies on a time-driven polling approach rather than a fully event-driven mechanism.

Overall, the integration of FINS into FUXA successfully fulfilled the two primary research objectives: (1) enabling the HMI system to communicate with Omron PLCs using the FINS protocol with functionality equivalent to existing Modbus support, and (2) improving the stability and visualization of data exchange on the HMI interface. Future research should focus on optimizing the alarm handling mechanism through event-driven or interrupt-based approaches, enhancing network security with TLS or VPN, and expanding multi-protocol interoperability. Additionally, open publication of the connector's source code, testing scripts, and datasets is encouraged to promote reproducibility and community-driven advancement in open-source industrial automation systems.

REFERENCES

- [1] Mohsin Ali Koondhar dkk. "The Role of PLC in Automation, Industry, and Education: A Review". In: *Pakistan Journal of Engineering Technology*

- and Science 11.2 (2023), pp. 22–31. doi: 10.22555/pjets.v11i2.975.
- [2] S. Mujawar. “Introduction to Human–Machine Interface”. In: *Handbook/Book on Human–Machine Interfaces*. Chapter 1; Accessed via Wiley Online Library. Wiley, 2023. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781394200344.ch1> (visited on 09/26/2025).
 - [3] Jyotika Sawal. “SCADA Market Size, Share, Trend Analysis by 2033”. <https://www.emergenresearch.com/industry-report/scada-market>. Report ID: ER_004544. May 2025. URL: https://www.emergenresearch.com/industry-report/scada-market?srsltid=AfmB0orSdQPJM_IH0PBeriQx0KYb023L8TR4JyuIII93M6iZJ4ojmwiu.
 - [4] Alisyn Gularte. “Open vs. Proprietary: Choosing the Right SCADA System for Your Solar Plant”. Blog post, Nor-Cal Controls. Accessed: 2025-09-12. Feb. 2025. URL: <https://blog.norcalcontrols.net/open-vs.-proprietary-choosing-the-right-scada-system-for-your-solar-plant>.
 - [5] Bakul Banthia. *Understanding the Risks of Cloud Vendor Lock-In*. Disaster Recovery Journal. Tessell. Aug. 13, 2024. URL: https://drj.com/industry_news/understanding-the-risks-of-cloud-vendor-lock-in/.
 - [6] McKinsey & Company. “Is industrial automation headed for a tipping point?” In: *McKinsey & Company Insights* (2023). URL: <https://www.mckinsey.com/capabilities/operations/our-insights/is-industrial-automation-headed-for-a-tipping-point>.
 - [7] Kasun Thushara. “Getting Start with FUXA - Web Based SCADA Tool”. Accessed: 2025-07-16. 2024. URL: https://wiki.seeedstudio.com/reTerminal-DM_intro_FUXA/.
 - [8] Davide Ancona dkk. “Towards Runtime Monitoring of Node.js and Its Application to the Internet of Things”. In: *arXiv preprint arXiv:1802.01790* (2018). doi: 10.48550/arXiv.1802.01790. URL: <https://arxiv.org/abs/1802.01790>.
 - [9] Joshua D. Scarsbrook, Mark Utting, and Ryan K. L. Ko. “TypeScript’s Evolution: An Analysis of Feature Adoption Over Time”. In: *arXiv preprint arXiv:2303.09802* (2023). URL: <https://arxiv.org/abs/2303.09802>.
 - [10] Peter Humaj. “Communication with Omron FINS”. Blog post on Ipesoft D2000. Apr. 2021. URL: <https://d2000.ipesoft.com/blog/communication-omron-fins/>.
 - [11] Brijesh Joshi. “A Study of the Various Communication Protocols Used in PLC Systems: Modbus, Profibus, Ethernet/IP and Their Implementations, Evolution and Comparative Analysis”. In: *International Research Journal of Modernization in Engineering Technology and Science (IRJMETS)* 6.4 (Apr. 2024). doi: 10.56726/IRJMETS52882. URL: https://www.irjmets.com/uploadedfiles/paper/issue_4_april_2024/52882/final/fin_irjmets1713198034.pdf.
 - [12] Sheikh Usman Uddin, Mirza Jabbar Aziz Baig, and Mohammad Tariq Iqbal. “Design and Implementation of an Open-Source SCADA System for a Community Solar-Powered Reverse Osmosis System”. In: *Sensors* 22.24 (2022), p. 9631. doi: 10.3390/s22249631. URL: <https://www.mdpi.com/1424-8220/22/24/9631>.
 - [13] Mohammad Omidi, Majid Salehifar, and Amir Mohammadi. “Design and Implementation of an Open Source SCADA System for Monitoring a Hybrid Renewable Power System”. In: *Energies* 16.5 (2023), p. 2229. doi: 10.3390/en16052229. URL: <https://www.mdpi.com/1996-1073/16/5/2229>.
 - [14] M. S. Almas and L. Vanfretti. “Open Source SCADA Implementation and PMU Integration”. In: *IEEE PES General Meeting* (2014).
 - [15] Luis Salazar dkk. “ICSNet: A Hybrid-Interaction HoneyNet for Industrial Control Systems”. In: *Proceedings of the Sixth Workshop on CPS&IoT Security and Privacy (CPSIoTSec’24)*. Salt Lake City, UT, USA: ACM, 2024, pp. 1–12. ISBN: 979-8-4007-1244-9. doi: 10.1145/3690134.3694813. URL: <https://doi.org/10.1145/3690134.3694813>.
 - [16] Robby Hazdi, Muhammad Fadli, and Yusril Maulana. “Perancangan SCADA pada Sistem Otomatisasi Rumah”. In: *eProceedings of Applied Science* 3.2 (2017).

Universitas Telkom, pp. 1099–1106. URL:
[https : / / openlibrarypublications .
telkomuniversity . ac . id / index . php /
appliedscience/article/view/4046](https://openlibrarypublications.telkomuniversity.ac.id/index.php/appliedscience/article/view/4046).